

2026 FreeRTOS

1. 任务通信

1.消息队列：实现任务间 / 任务与中断间的任意长度数据传递

- (1) 相当于一个独立于任务外的缓冲区，**支持多发送方、多接受方**
- (2) 数据可以**选择拷贝数据传递或者传递指针**，读写队列时会进入临界区，防止多任务同时访问冲突
- (3) 支持 FIFO/LIFO 模式，可以配置队列长度和队列项大小，队列元素大小不宜过大，拷贝开销大
- (4) 接收方可设置阻塞超时，发送方也可设置 队列满时的阻塞策略

应用场景：比如 “串口接收任务” 将接收到的字节数据通过队列传给 “数据解析任务”，避免全局变量竞争

2.信号量：同步与资源计数，解决时序/资源竞争

- (1) 二值信号量：任务/中断同步

只有0/1两种状态，用于事件通知，可用于中断通知任务

- (2) 互斥信号量：临界资源保护

支持优先级继承：当低优先级任务持有 Mutex，高优先级任务请求该 Mutex 时，低优先级任务的优先级会被提升到高优先级任务的级别，直到释放 Mutex，避免 “优先级翻转”

不能在中断中使用，也不能在持锁时调用阻塞函数

创建互斥信号量时，会主动释放一次信号量

- (3) 计数信号量：—— 资源计数

用于对 “有限资源的访问控制” ；

应用场景：限制同时访问 SPI 总线的任务数（如最多 2 个任务）

3.事件标志组：多事件同步

支持 “等待任意事件（逻辑或）” “等待所有事件（逻辑与）”

- (1) 支持 “事件位清除策略”：可以设置事件触发后清除对应位或保留
- (2) 中断中可设置事件位
- (3) 不用于传递大量数据

应用场景：系统初始化（等待串口、I2C、网络都初始化完成）

4.消息缓冲区 / 流缓冲区

采用 “零拷贝” 或 “少拷贝” 模式，适合串口、网口等 “不定长字节流” 场景；

- 消息缓冲区：按“消息边界”传递（如每帧数据带长度），适合离散消息；
- 流缓冲区：按“字节流”传递（无边界），适合连续数据（如文件传输）；

5.任务通知：轻量级通信

直接向任务发送通知（无需创建独立的内核对象，每个任务默认有一个 32 位的“通知值”），开销小，但无法广播给多个任务，支持中断发送通知给任务

（1）发送任务通知，可配置为 4 种模式：

- ☒ **简单通知**：仅通知任务
- ☒ **带值通知**：传递 32 位数值
- ☒ **递增通知**：通知值自增
- ☒ **位操作通知**：按位设置 / 清除

发送受阻不支持阻塞，发送方无法进入阻塞状态等待

（2）接收任务通知

通知可“留待处理”：若任务未就绪，通知会被保存，任务就绪后立即处理。

6.队列集：解决“一个任务等待多个队列 / 信号量”的痛点

队列集允许将多个队列 / 二进制信号量加入一个“集合”，任务只需等待这个集合，就能响应任意一个队列 / 信号量的事件。

任务等待队列集时，只要集合中有任意一个队列有数据 / 信号量可用，就会被唤醒，**满足多事件中单个事件触发一个任务**

2. 内存管理

FreeRTOS提供了5种动态内存管理算法

1.heap_1：只实现了pvPortMalloc，没有实现vPortFree，只能申请内存，无法释放内存

2.heap_2：使用最适应算法，并且支持释放内存，但并不能将相邻的空闲内存块合并成一个大的空闲内存块

最适应算法：根据需要申请的内存大小，找出最小的且能满足内存要求的内存块

3.heap_3：调用C库函数malloc()和 free()，Heap_3中先暂停FreeRTOS的调度器，再去调用这些函数，使用这种方法实现了线程安全

4.heap_4：使用了**首次适应算法**，也支持内存的申请与释放，并且**能够将空闲且相邻的内存进行合并**，从而减少内存碎片的现象

5.heap_5：在 heap_4 内存管理算法的基础上实现的，但是 heap_5 内存管理算法在 heap_4 内存管理算法的基础上实现了管理多个非连续内存区域的能力

heap_4内存管理算法：

1.定义一个大数组作为heap_4管理的**内存堆**

2.初始化内存堆：prvHeapInit()

3.内存申请：pvPortMalloc()，遍历**内存块链表**，找到第一个内存大小合适的内存块，将返回值设置为可分配内存的起始地址

4.内存释放：vPortFree()，**将释放的内存块插入空闲列表中**

5.**空闲内存链表**的插入：prvInsertBlockIntoFreeList()，用于将空闲内存块插入空闲块链表，由地址从低到高排列，并将相邻的两个内存块合并

3. 任务调度/启动第一个任务

1.任务调度触发：发生时间片轮转或者抢占后，调度大多由 **SysTick 中断** 触发

2.调度策略

(1) **抢占式调度**：更高优先级任务变为就绪态，立即抢占当前任务

(2) **时间片轮转调度**：相同优先级的任务按照顺序在时间片结束后轮流执行，每个任务分配一段固定的时间片

3.SVC启动第一个任务

(1) **获取当前优先级最高的就绪态任务的任务控制块**

通过 pxCurrentTCB 获取优先级最高的就绪态任务的任务栈地址（第一个元素为栈顶）

pxCurrentTCB 是一个全局变量，用于指向系统中优先级最高的就绪态任务的任务控制块

(2) **将该任务的寄存器值出栈至CPU寄存器中**

通过任务的栈顶指针，将任务栈中的内容出栈到CPU寄存器

(3) **设置PSP**，允许中断

设置PSP指向该任务栈

往BASEPRI寄存器写0，允许中断

(4) **返回R14**，使CPU跳转到任务的函数中去执行

R14（链接寄存器LR）：用于保存函数的返回地址

4. 任务切换/上下文切换

1.任务切换触发

(1) **SysTick中断**触发（设置PendSV异常挂起标志）

(2) 调用FreeRTOS的API函数执行系统调用（portYIELD()）

2.上下文切换

(1) 自动保存现场：PendSV响应，硬件自动入栈（PSP此时指向任务A）R0,R1,R2,R3,PSR,LR,PC

(2) 执行PendSV：（取向量，更新寄存器，执行异常服务例程）

1.保存现场：保存CPU中寄存器的值到A的任务栈

读取psp，手动保存 r4-r11，将新栈顶写回到当前任务的TCB

2.获取下一个运行的任务，更新pxCurrentTCB

调用 `vTaskSwitchContext` 获取下一个运行的任务，更新指向当前任务的任务控制块变量

（M3中使用特殊方法即使用**读取前导0的指令**读取记录任务优先级的变量_clz，从而得到当前最高的优先级）

3.恢复现场，更新psp：将任务B中寄存器的值重装到CPU的寄存器

从新的TCB读出栈指针，手动恢复R4-R11，更新PSP为任务切换后的任务栈指针

4.BX R14 （从异常返回）

异常返回值：在进入异常服务程序后，LR的值被自动更新为特殊的**EXC_RETURN**（高27位全为1的值，只有[5:2]的值有特殊含义）

（3）中断返回，硬件自动入栈（PSP此时指向任务B）R0,R1,R2,R3,PSR,LR,PC，（更新NVIC寄存器），CPU跳转到PC指向的代码位置运行

5. 低功耗

1.STM32低功耗模式

（1）sleep睡眠模式：CPU停止运行，外设时钟未关闭

_WFI/WFE指令进入，可在发生中断/事件时唤醒CPU，唤醒事件最短

（2）stop停止模式：在保持SRAM和寄存器内容不丢失的情况下，停机模式可以达到最低的电能消耗可以通过任一配置成**EXTI**的信号唤醒，EXTI信号可以是16个外部IO口之一、PVD的输出、RTC闹钟或USB的唤醒信号

置位系统控制器相关位->清零电源控制器相关位->配置外部中断线唤醒->WFE指令进入

（3）standby待机模式：SRAM和寄存器内容不保留，但后备寄存器内容仍然保留，待机电路仍工作达到最低的电能消耗，从待机模式退出的条件是:NRST上的外部复位信号、IWDG复位、WKUP引脚上的一上升边沿或RTC的闹钟到时

2.Tickless

进入空闲任务->挂起任务调度器，计算睡眠时间->重装载滴答定时器->调用_WFI指令进入睡眠模式->时间补偿->恢复任务调度器

重装值=当前的滴答定时器计数值+待运行时间×一个时钟节拍的 reload 值（168M/1000）

（1）如果小于最大睡眠时间，停止滴答定时器 SysTick，要根据待运行任务的阻塞时间去重新计算滴答定时器的重装值reload，这样在时间到达以后会触发滴答定时器中断，即可从睡眠模式中唤醒，达到应用层任务唤醒睡眠模式的目的

($ulTimerCountsForOneTick=168M/1000$ ，因为滴答定时器频率为1000Hz，则一次节拍的reload值为168M/1000)

(2) 当发生中断时，需要去判断是滴答定时器中断唤醒还是其他中断唤醒

如果是滴答定时器唤醒的，补偿时间为任务待运行时间

如果是其他中断唤醒的，补偿时间为实际消耗时间